

Defect Prediction With Semantics and Context Features of Codes Based on Graph Representation Learning

Jiaxi Xu¹, Fei Wang¹, and Jun Ai¹

Abstract—To optimize the process of software testing and to improve software quality and reliability, many attempts have been made to develop more effective methods for predicting software defects. Previous work on defect prediction has used machine learning and artificial software metrics. Unfortunately, artificial metrics are unable to represent the features of syntactic, semantic, and context information of defective modules. In this article, therefore, we propose a practical approach for identifying software defect patterns via the combination of semantics and context information using abstract syntax tree representation learning. Graph neural networks are also leveraged to capture the latent defect information of defective subtrees, which are pruned based on a fix-inducing change. To validate the proposed approach for predicting defects, we define mining rules based on the GitHub workflow and collect 6052 defects from 307 projects. The experiments indicate that the proposed approach performs better than the state-of-the-art approach and five traditional machine learning baselines. An ablation study shows that the information about code concepts leads to a significant increase in accuracy.

Index Terms—Deep learning, defect prediction, graph representation learning, software defect dataset, software engineering.

I. INTRODUCTION

SOFTWARE defect prediction (SDP) identifies defect-prone modules in computer programs, thereby optimizing the software testing process and improving the quality and reliability of software. In recent years, machine learning has been widely used for SDP [1]–[4]. The practitioners of traditional SDP hold that defects are related to module size and complexity, and have developed metrics, such as McCabe cyclomatic complexity, Halstead complexity measures, CK metrics, and MOOD. These metrics and algorithms help software researchers better predict defects. For example, Ceylan *et al.* [5] employed decision tree learning, a multilayer perceptron neural network, and radial basis functions to predict the number of total defects per module. Tong *et al.* [6] enhanced denoising autoencoders for defect classification on class imbalanced datasets.

Manuscript received July 1, 2020; revised October 9, 2020; accepted November 19, 2020. Date of publication December 10, 2020; date of current version June 1, 2021. Associate Editor: J. Zhang. (Corresponding author: Jun Ai.)

The authors are with the School of Reliability and System Engineering, Beihang University, Beijing 100191, China (e-mail: hickeyhsu@buaa.edu.cn; wangfeiabn@buaa.edu.cn; aijun@buaa.edu.cn).

Color versions of one or more of the figures in this article are available online at <https://doi.org/10.1109/TR.2020.3040191>.

Digital Object Identifier 10.1109/TR.2020.3040191

The amount of data plays a critical role in machine learning, since training with more data can make models generalize better. Public defect data repositories, such as NASA-MDP and PROMISE, are essential for making unbiased comparisons of different approaches and have contributed to the development of SDP. The existing research has been dominated by defect metrics and learning models using publicly available datasets [7] but it has ignored the data that are widely available from open-source software (OSS). Recent developments in the field of OSS have led to a massive increase in the availability of data [8] due to open-source programs exposing the source code as well as software descriptions, bug fixes, and review processes. The hope is that deeply hidden defect patterns can be uncovered by optimizing the information available from OSS.

Defect representation is also a significant problem in mining OSS repositories for the defect prediction. Programs have well-formatted syntax, which can be represented by abstract syntax trees (ASTs). Many studies have leveraged ASTs to mine code semantics in software engineering domains (e.g., defect prediction, clone detection, and program synthesis), called *learning from big code* [9], [10]. However, a software defect is related not only to the syntax and semantics but also to software requirements, functions, and dependencies. For instance, two identical code modules can be defect-prone or nondefect prone depending on their environments. To date, there have been no firm conclusions regarding how to capture defect features to build a more accurate SDP model.

In this article, we proposed a novel defect prediction approach that identifies defect patterns in different environments. Instead of using the traditional defect metrics, software defects were characterized by defect subtrees of ASTs that integrate the information of fix-inducing change and code concepts. A topic model was developed to summarize the functional concepts around defects. Accordingly, each node of the defect subtrees contains the attributes of types, fix-inducing change, and code concepts. Inspired by the recent success of graph neural networks (GNNs), defect subtrees were treated as directed acyclic graph structures, and the GNNs were leveraged to learn the graph representation. With different graph-level representations of defect and nondefect, the graph classification task was defined for building an SDP model.

To collect defect data from an open-source project in GitHub, we queried git repositories for defect-related pull requests (PRs) and then obtained class-level samples from the workflow of code

changes. If a defect was successfully fixed, the class before being fixed was treated as a defect sample, and the class after being fixed was treated as a nondefect sample. Therefore, the number of defect samples is the same as the number of nondefect samples in our GitHub pull request (GHPR) dataset. We conducted empirical experiments to confirm the effects of the proposed method on the GHPR dataset, which contains 307 software projects and 6052 defect instances. Accuracy, area under the curve (AUC), and F1 score were used as the evaluation criteria. Experiments were conducted to compare the proposed method with machine learning models based on CK metrics and the deep belief network cross-project defect prediction (DBN-CP) method proposed by Wang *et al.* [7]. The results showed a significant improvement of the classifier performance in the proposed method. The three major contributions of this article can be summarized as follows.

- 1) Creating a public GHPR dataset containing thousands of buggy and clean sample pairs, which solves the imbalanced classification problem of the defect prediction.
- 2) Proposing a novel defect representation with the capability of capturing the semantic and context information.
- 3) Introducing GNNs to the process of SDP. The experimental results showed that applying GNNs with the proposed defect representation can improve the state-of-the-art of classification results.

The remainder of this article is organized as follows. Section II offers some background on the defect prediction and GNNs. Section III presents our defect prediction approach using GNNs. Section IV describes an experimental study using our GHPR dataset. Sections V–VI compare our work with the state-of-the-art approach. Finally, Section VII concludes this article.

II. BACKGROUND

A. Software Defect Prediction

SDP is an increasingly important area in software engineering that uses historical data to establish the relationship between software metrics and defects. The goals of SDP are to estimate the number of defects, identify defect patterns, and classify the defect proneness of software modules [11]. This study focuses on the last of these three tasks. SDP typically contains four steps, i.e., data preprocessing, metrics selection, model learning, and model evaluation.

SDP is formally defined as a binary classification problem. Let $D = \{(x_i, y_i)\}_{i=1}^P$ be a training dataset, where P is the number of samples, and y_i is the label of sample x_i . In the defect prediction problem, $y_i \in \{0, 1\}$, where $y_i = 1$ indicates that the module contains one or more defects and $y_i = 0$ indicates a module without defect proneness. Furthermore, $x_i \in \mathcal{R}^M$ represents defective modules as characterized by M software metrics. A target dataset is defined as $U = \{u_i\}_{i=1}^Q$, where Q is the number of samples to be predicted. Sample u_i is also characterized by the same M software metrics. The goal of the defect prediction model is mapping u_i to targets $\{0, 1\}$. Accordingly, SDP can be viewed as a process of finding the optimal mapping function $f(u_i)$.

B. Graph Neural Networks

GNNs have been successfully adopted in applications, such as social network analysis, biological information, image recognition, and human action recognition. GNNs are the graph-embedding methods with neural networks. More formally, a graph that contains N nodes is represented as $G = (V, E)$, where $V = \{v_1, v_2, v_3, \dots, v_N\}$ and $E = \{e_{ij} = (v_i, v_j) | i, j = 1, 2, 3, \dots, N\}$. Let X_v , for $v \in V$, denote a node feature matrix.

GNNs have two crucial tasks: node classification and graph classification. For the node classification, GNNs learn a representation of each node by aggregating its neighbors. The updated rule [12] from the l layer to the $l+1$ layer is as follows:

$$\Theta_v^{(l+1)} = \text{AGGREGATE}^{l+1} \left(H_u^{(l)} | u \in \mathcal{N}(v) \right) \quad (1)$$

$$H_v^{(l+1)} = \text{COMBINE}^{l+1} \left(H_v^{(l)}, \Theta_v^{(l+1)} \right) \quad (2)$$

where $\Theta_v^{(l)}$ and $H_v^{(l)}$ are the aggregation weights and the underlying representation of node v in the l layer, respectively. $\mathcal{N}(v)$ is an unordered set of neighbors of node v . For the first layer, $H_v^{(0)} = X_v$. The specific models are only developed by designing the permutation invariant $\text{AGGREGATE}^l(\cdot)$ and $\text{COMBINE}^l(\cdot)$. After the parameters are trained iteratively, the model encodes the structural information into vectors for the node classification and link prediction.

For the graph classification, the readout operation aggregates the node features to generate a graph-level representation. The conventional method creates a simple statistic of the representation of each node in the final layer, such as sum, average, or max pooling. The graph-level representation is calculated as

$$H_G = \text{READOUT} \left(\left\{ H_v^{(L)} | v \in V \right\} \right). \quad (3)$$

Here, H_G is the representation for graph G and L is the final layer of GNN. In general, fully connected layers connect every neuron of the last layer to obtain all the information for learning. In this article, ASTs of the source code can be considered as a graph lying in an irregular domain. We believe that GNNs are powerful enough to capture the latent defect information in ASTs.

III. METHODOLOGY

This section describes the proposed method in detail. Fig. 1 illustrates the overall framework of our proposed approach. The framework is divided into the following four steps.

- 1) In step one, the defect data were collected by mining PRs of GitHub.
- 2) In step two, for each defective instance, the source code was parsed into structured semantics (ASTs) and redundant information was discarded.
- 3) In step three, code concepts were extracted to build a high-level understanding of the defective snippets.
- 4) Finally, in step four, GNNs were adapted to learn a latent representation of defective semantics and concepts.

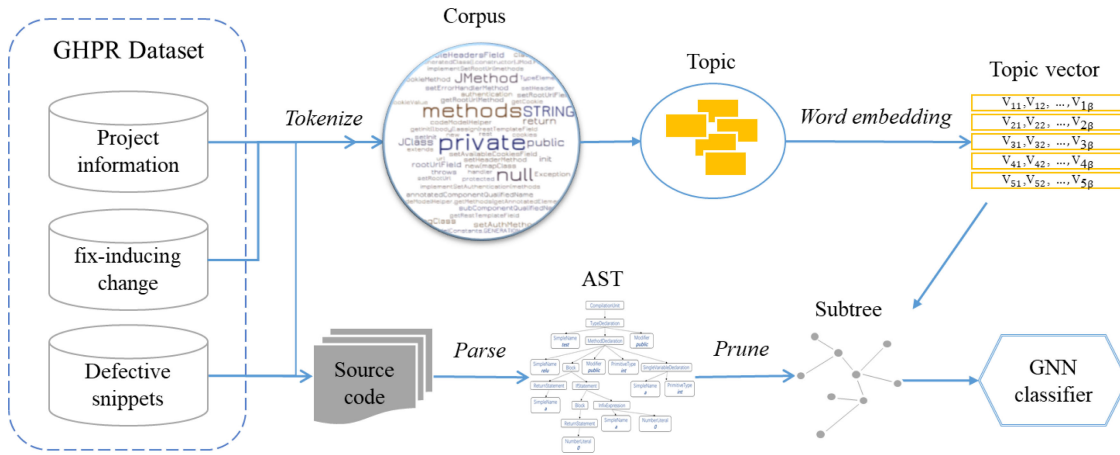


Fig. 1. Overview of the proposed approach.

A. Data Collection

To collect the high-quality data about defects, reliable defect information was identified based on the PRs of the GitHub workflow. Previous studies identified defective source code parts by git commits and issues on GitHub but many descriptions of commits (or issues) are not well formed, which adds noise to defect datasets. The GitHub workflow is a stricter branch-based model for feature implementations and bug fixes. In the GitHub workflow, developers create a new branch and cause changes. A PR is opened to initiate a review and notify the maintainers of the changes. Each PR can be labeled as “bug,” “enhancement,” “feature,” etc. When the changes are verified by the project maintainers, the branch will be merged into the master branch. We assume that the defect information with team review and the GitHub workflow is more accurate. Generally, the mining process consists of three steps: project selection, defect identification, and information extraction.

1) *Selecting the Projects*: There are over 100 million open repositories on GitHub. Applying two criteria, a subset of git repositories was selected. The first criterion was that projects must have at least 1000 forks. The number of forks is related to the activity of the PRs in a repository since the contributors usually fork the repository before creating a PR. The second criterion was that projects must be Java projects, in which PRs are described in English. This criterion gives us identifiable defects using general regular expression rules.

2) *Identifying Defects*: For each project, PRs were extracted and matched with regular expression rules. The regular expression rules cover various defect expressions and keywords (e.g., bug, defect, or fix). The selected PRs must be closed and merged into the master branch, which shows that the fixes have been verified by team review. After finding the PRs related to defects, we can obtain two source code file versions by analyzing the GitHub workflow, which is illustrated in Fig. 2. Each PR contains three crucial commits: base commit, head commit, and merge commit. The base commit is pulled before creating a new branch, the head commit is the last commit in the branch, and the merge commit is the earliest commit after merged. The

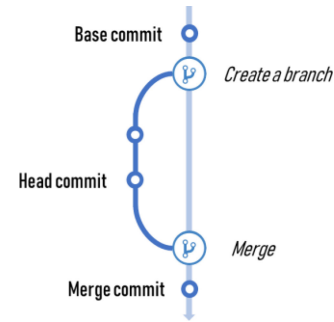


Fig. 2. GitHub workflow.

defective version and the fixed version were obtained based on the base commit and merge commit. Note that the head commit is a candidate for the merge commit, which could be lost in the GitHub application programming interface (API).

3) *Extracting Information*: Each defect contains project information, fix-inducing change, and source code. Project information and fix-inducing changes, such as a project name, label, and fix-inducing change description, were obtained from the git remote repository with the GitHub API. Based on the defective and fixed versions identified above, the source code was extracted from the local repositories using the JGit, an implementation of Git written natively in Java. Additionally, single-point defects, in which only one class file had been changed, were collected.

B. Defect Representation

At the heart of SDP is the need to create the proper defect representation. Previous studies focused on the artificial software metrics, viewed from the complexity and relevancy of modules. The traditional metrics have two major drawbacks. First, it is difficult for researchers to design a set of metrics and validate it. Second, the metrics ignore the syntactic and semantic information that may be effective for improving the performance of the defect prediction. The practices for software development

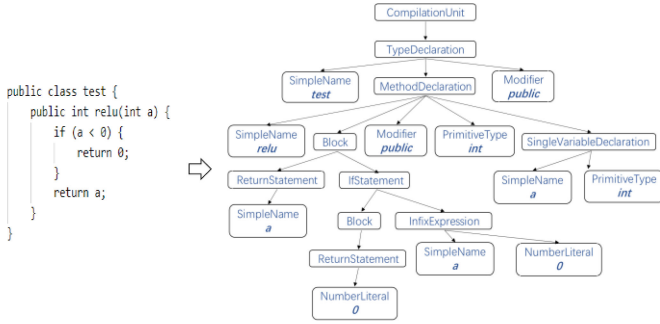


Fig. 3. AST parsing example.

also indicate that the semantic and context information is often the essential evidence for finding defects.

1) *ASTs Extraction*: To extract the semantic information, ASTs were used as the code representation, which is widely explored in the program analysis. A number of recent articles [7], [13]–[16] have provided a strong evidence for the advantages of ASTs in the source code modeling, arguing that they can be used to obtain better code representation than the traditional methods. AST is a type of tree-based structure for representing the syntax and semantics of the source code, in which every node represents a syntax structure. Meanwhile, ASTs contain actual characters, such as brackets, which is the “abstract” in its name. Fig. 3 shows a simple AST example parsed from the Java source code.

Each compilation unit is parsed into an AST that is a labeled, ordered, rooted tree. The root of ASTs is the compilation unit, and every node contains a parent and a sequence of children. In particular, the root has no parent, and the leaves have an empty set of child. There are directed edges between the parent and its children. The ASTs of Java contain 88 node types, such as MethodInvocation, IfStatement, and SimpleName. Since the text feature cannot be provided to learning models, a mapping encoder is built between node types and integers. The encoder converts each type into a unique identifier.

If the change code in the bug fixes is defect-related, mismatched nodes between the buggy AST and the fixed ASTs can be used to represent the defect-related information. The fixed ASTs are regarded as being transformed from the buggy ASTs through a sequence of edit operations, the minimum cost sequence of which is called *tree edit distance*. We leveraged a freely available tool *GumTree* to calculate the tree edit distance of two trees [17] and considered the nodes in edit scripts as mismatched nodes. When the change code only accounts for a low percentage of all the compilation units, the mismatched nodes are only a tiny fraction of the complete ASTs, which contain thousands of nodes. In these cases, the defect patterns are drowned in redundant data, which leads to a high similarity of features between the buggy AST and the fixed AST.

2) *AST Pruning*: The AST generated by the above-mentioned steps can range from dozens to thousands of nodes, depending on the size of the code file. In the dataset we collected, the largest AST had 2684 nodes, of which only a few dozen were modified before and after the defect repair. Therefore, a lot of

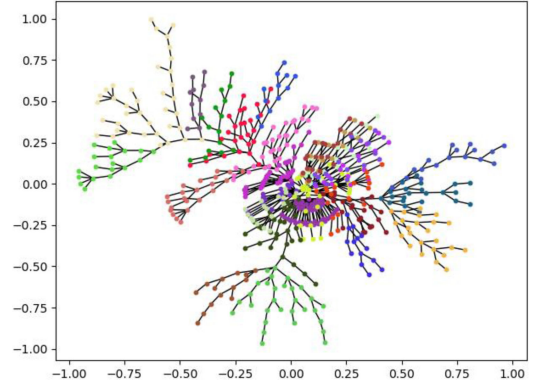


Fig. 4. AST after community division.

redundant information and noise are introduced into the sample. To solve this problem, we need to prune the AST. However, the defect subtree obtained directly by cutting off all the nodes other than the modified node is a highly discrete and incomplete defect syntax structure, so the nodes closely related to the modified node need to be retained.

To prune ASTs regarded as undirected acyclic graphs, this article implemented the Louvain algorithm of community detection [18], which extracts defect-related communities from a large tree. The method reveals a hierarchy of communities by maximizing the modularity scores of communities. The modularity scores are calculated by

$$Q = \frac{1}{2m} \sum_{i,j} \left[A_{i,j} - \frac{k_i k_j}{2m} \sigma(c_i, c_j) \right] \quad (4)$$

where $A_{i,j}$ is the number of edges between node i and node j , k_i and k_j are the number of edges adjacent to i and j , m is the total number of edges in ASTs, and $\sigma(c_i, c_j)$ is an indicator function, which is as follows:

$$\sigma(c_i, c_j) = \begin{cases} 1 & \text{if } i \text{ and } j \text{ are in the same community} \\ 0 & \text{otherwise} \end{cases} \quad (5)$$

The community detection groups ASTs into several distinct subtrees of a relatively high density. Based on the results of this method, ASTs are pruned to the defect-related subtrees, whose nodes contain mismatched nodes. If these communities cannot form a connected subtree, they are connected by their respective roots. Algorithm 1 shows the details of pruning ASTs to the defect-related subtree. Generally, in our defect-related subtree, defect information is accurately extracted by the code change information, and grammatical structures are reserved by the Louvain algorithms.

The AST after community division is shown in Fig. 4. As can be seen from Fig. 5, after the AST is clipped using the Louvain algorithm, the defect subtree contains only defect repair nodes and nodes closely connected to the syntax structure. Many redundant nodes in the original syntax tree are effectively filtered. A defect subtree can be thought of as a context syntax structure information within the limited scope of a defect change code.

Algorithm 1: Pruning ASTs Into Defect-Related Subtrees.

Input: A defective AST T_0 , a fixed AST T_1 from T_0 ,
 matched nodes $N_i = \{n_{i1}, n_{i2}, \dots, n_{ij}, \dots\}$ in T_i using
GumTree, the communities $C_i = \{c_{i1}, c_{i2}, \dots, c_{ij} \dots\}$
 for T_i using LDA
 Output: Defect-related subtrees T_0^* and T_1^*

```

1 Initialize list  $V_0, V_1, M_1$ , and  $M_2$ 
2 For  $i \in [1, 2]$ 
3   If node  $n \in c_{ij}$  and  $n \notin N_i$  then
4     Add  $c_{ij}$  into  $V_i$ 
5   EndIf
6 EndFor
7 For  $i \in [1, 2]$ 
8   If  $V_i = \text{null}$  then
9     If node  $n \in c_{(2-i)j}$  and  $n \in N_{(2-i)}$  then
10      Add  $n$  into  $M_{(2-i)}$ 
11    EndIf
12    Mapping  $M_{(2-i)}$  to  $M_i$ 
13    If node  $n \in c_{ij}$  and  $n \in M_i$  then
14      Add  $c_{ij}$  into  $V_i$ 
15    EndIf
16    For each  $c_{ij} \in V_i$  do
17       $T_i^* \leftarrow \text{connect}(T_i^*, c_{ij})$ 
18    EndFor
19 EndFor

```

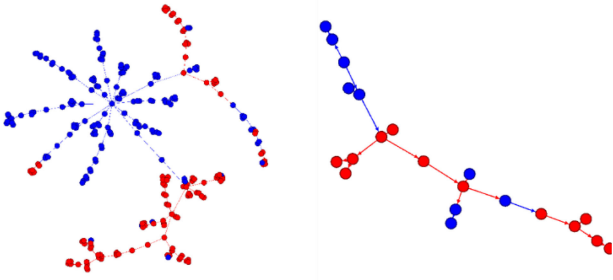


Fig. 5. Defect subtree after pruning.

C. Extraction of Code Concepts and Context Concepts

After structuring the code in Section III-B, we obtained the AST that represents the structure of the code, where each node contains attributes that represent the type of the node. Software defects are related not only to the syntax and structure but also to the software functions of semantics and context. Even two identical pieces of code may differ in their defect proneness in different environments. Based on conceptual concerns, the topic model was used to capture a high-level meaning of the source code from semantics, project information, and fix-inducing changes. Project information and fix-inducing changes in the form of natural language can be obtained directly in our dataset but semantics need to be extracted from names, types, and comments in the defective source code.

1) *Data Preprocessing*: The concept features of each defective module correspond to the bag of words model, including semantics and contexts. The semantic concept refers to the

TABLE I
REGULATIONS OF NODES TYPE FOR DATA IDENTIFICATION

Data	Regulation
class name	CompilationUnit-TypeDeclaration-SimpleName
member variable type	CompilationUnit-TypeDeclaration-FieldDeclaration-VariableDeclarationFragment-SimpleType-SimpleName
member variable name	CompilationUnit-TypeDeclaration-FieldDeclaration-VariableDeclarationFragment-SimpleName
method name	MethodDeclaration-SimpleName
formal parameter type	MethodDeclaration-SingleVariableDeclaration-SimpleType-SimpleName
formal parameter name	MethodDeclaration-SingleVariableDeclaration-SimpleName
method return types	MethodDeclaration-SimpleType-SimpleName
variable name	VariableDeclarationStatement-VariableDeclarationFragment-SimpleName

semantic information extracted from the syntax, variable name, method name, and so on in the code. The context concept refers to the meaningful information extracted from project information and fix-inducing changes. This section synthesizes this information into background information, treats it as an integrated document, processes it through the natural language processing method to generate code topic vectors, and embeds the vectors in each node of the AST as conceptual attributes.

Based on the structure of ASTs, this article identified several semantics-related categories, including variable names, method names, return types, etc. The semantics-related categories were extracted by the regulation of the node type in ASTs, as given in Table I. Since the names and types are often composed of more than one word, they were processed according to universal naming conventions. For example, *GetRequestToken* was split into three words—*Get*, *Request*, and *Token*—according to the standard CamelCase syntax. In addition, stop words that contained unimportant information were filtered out as well as the keywords in programming, such as *main* and *args*.

The context information is mainly composed of English natural language description. According to the space, punctuation, and paragraph, the text information was divided into words. Then, a filtering process similar to the processing for the semantic information was performed. The concept information was obtained by merging the context information with the semantic information.

2) *Topic Model*: To generate a set of topics for the concept information, our approach uses the latent Dirichlet allocation (LDA) model, a generative statistical model that identifies topics to represent text documents. Chen *et al.* [19] used the LDA modeling technique to design metrics for explaining defect proneness. We used the *scikit-learn* machine learning library to implement the LDA model with the expectation-maximization method of online variational Bayes. Taking into consideration the semantics and corpus size, the number of topics was set to one in a defective module. The top- α words with the highest probability were used to represent the code concepts.

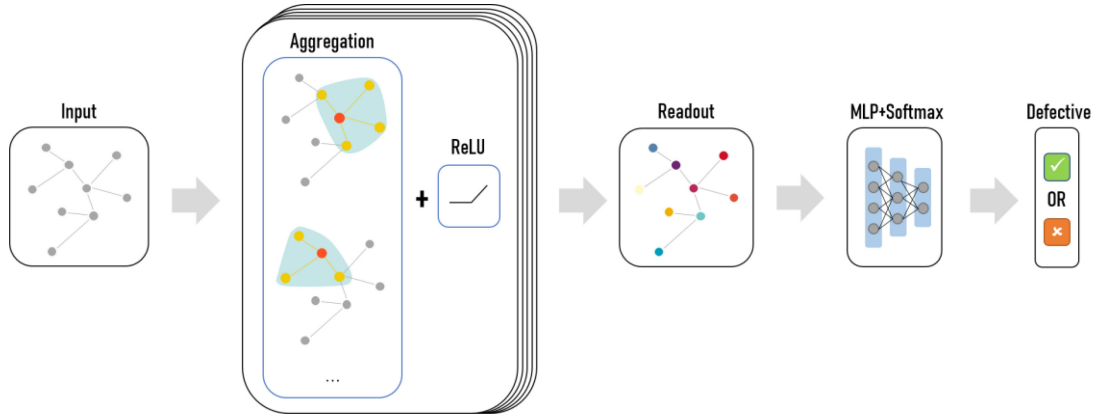


Fig. 6. GNN architecture.

3) *Word Embedding*: To achieve learning with the code concepts, topics were converted into a numeric vector, which was called *word embedding*. Based on the distributed hypothesis, the word2vec model develops a two-layer neural net in order to give similar representations of words with similar meanings [20]. Each word of topics was mapped into a β -dimensional vector so that a concept matrix $\mathcal{M}$ is an $\alpha \times \beta$ matrix. In addition, a word2vec model fully supporting the incremental training was implemented because the model can only return a representation of the word in the vocabulary. Each input topic was treated as a sentence to update the vocabulary of the model. The topic vector is the expression of the conceptual information of the entire defect module so all nodes share the same topic vector as the conceptual attribute.

D. Defect Representation Learning

With the defect representation in hand, a supervised graph representation learning based on GNNs was defined. The input of defect representation learning is a defect-related subtree (as shown in Section III-B) combined with concept features. The subtrees were treated as undirected acyclic graphs, and concept features were attributes of the graph. The output of the end-to-end framework is either defective or not defective for each graph. The concept features $\mathcal{F}$ were represented as a vector compressed by the concept matrix (as shown in Section III-C), as follows:

$$\mathcal{F} = \frac{1}{\alpha} \sum_{i=1}^{\alpha} \mathcal{M}_i | \mathcal{M} \in \mathcal{R}^{\alpha \times \beta}. \quad (6)$$

The graph-level concept features were embedded into nodes as node-level attributes. The node attributes included node types parsed in ASTs and identical concept features in each graph.

GNNs framework and its variants [12], [21]–[24] transform the feature representation of nodes with the aggregation weights matrix, etc., and realize a nonlinear transformation using a rectified linear unit (ReLU) activation function. The structural information of subtrees and the node attributes were aggregated with a multilayer network comprised of one or more hidden layers. To simplify the model, each hidden layer has the same number of neurons. Taking graph convolutional networks (GCNs) [21] as an example, AGGREGATE() of Algorithm 2 shows the process

of the aggregation of the subtrees obtained in Algorithm 1 into the graph-level representation.

After obtaining the representation with a fixed number of dimensions for each node, max-pooling operations (a type of readout) were used to construct a graph-level representation. Two fully connected layers were able to integrate all the information into neural networks. Based on the obtained latent characteristics, the softmax function helped predict the defective samples. To avoid overfitting, the regularization technique known as dropout was used to drop a percentage of neurons in every forward propagation and weight updating. Many studies have shown that the temporary inactivation of partial neurons results in better generalization. The percentage of dropping was selected in Section IV-E, together with the number of layers and hidden size. Fig. 6 shows that the input defect-related subtree of the irregular structure is embedded for SDP. As showed in Algorithm 2, our prediction model is represented as function PREDICT(), which takes as an input a target sample subtree T achieved through Algorithm 1 and returns 1 if the subtree is defective; otherwise, it returns 0.

Although we use GCNs as an example in ALGORITHM 2, many different neighborhood aggregations or message passing schemes have been developed on GNNs. We used PyTorch Geometric, a popular geometric deep learning extension library for PyTorch, to implement GCN, graph isomorphism network (GIN) [12], simplified graph convolution (SGC) network [22], graph attention networks (GATs) [23], and GraphSAGE (SAGE) [24].

IV. EXPERIMENTS

A. Research Questions

Listed in the following part are the four research questions for our article .

- Q1. Which GNN model achieves the best performance in our proposed approach?
- Q2. Does the proposed approach improve the performance of the traditional machine learning model based on static features?
- Q3. Does the proposed approach outperform the state-of-the-art approach?
- Q4. How effective is the code concepts information?

Algorithm 2: Defect Representation Learning.Input: Trained GCNs model, target sample subtree T .

```

1  function PREDICT (Subtree  $T$ )
2    graph-level representation  $V \leftarrow \text{AGGREGATE}(T)$ 
3     $\hat{p} \leftarrow \text{MLP\_SOFTMAX}(V)$ 
4    If  $\hat{p} \geq 0.5$  then
5      return 1
6    Else
7      return 0
8    EndIf
9  End Function
10 function AGGREGATE ( $T$ )
11    $T_p \leftarrow \text{DATAPROCESSING}(T)$ 
12   For each node  $N \in T_p$ 
13      $t_N \leftarrow \text{LDA}(N)$ 
14      $\mathcal{M} \leftarrow \text{WORD2VEC}(t_N)$ 
15   EndFor
16    $\mathcal{F} \leftarrow \frac{1}{\alpha} \sum_{i=1}^{\alpha} \mathcal{M} | \mathcal{M} \in \mathcal{R}^{\alpha \times \beta}$ 
17    $V \leftarrow \text{GCNs}(\mathcal{F})$ 
18   return  $V$ 
19 End Function

```

Q1 aims to identify the best performer among five GNN models. Q2 and Q3 focus on the effectiveness of the proposed approach. Q4 deals with an ablation study for code concepts.

B. GHPR Dataset

We implemented the tool *BugMiner* [25] to automatically generate the GHPR dataset, which identified 3026 bug-fixing from 307 Java projects. Each bug-fixing consisted of 16 features of project information, fix-inducing change, and source code, as given in Table II. From the view of supervised learning, we considered the two of the defective and fixed file versions in each bug-fixing as two learning instances. The dataset contained 3026 defective instances and 3026 nondefective instances, for a total of 6052 learning instances. Although a defect tends to be related to multiple source files, only instances of single-point defect that all fixes are in one file were considered. The dataset was expected to achieve a high precision rather than a high recall because of the strict rules. Although only a small number of defects were needed to meet our requirements in a software project, the number of samples in our dataset was still considerable due to the vast number of projects in GitHub. Each sample was represented as a defective subtree. The largest subtree contained 2704 nodes, while the smallest contained five nodes. Our GHPR dataset is available in a GitHub repository.¹

C. Experimental Setting

Five GNNs (GIN, GCN, SGC, GAT, and SAGE) were utilized. All five GNNs have the same structure but differing neighborhood aggregation methods. After each convolution layer, we added an ReLU activation function, a batch normalization

TABLE II
GHPR DATASET FEATURES

Features	Description
PROJECT_NAME	The name of the project
PROJECT_OWNER	The owner of the project
PROJECT_DESCRIPTION	The description of the project provided by the owner
PROJECT_LABEL	The label of the project provided by the owner
PROJECT_LANGUAGE	The programming language of the project
SHA_FIXED	The ID of the version after the defect being fixed
SHA_BUG	The ID of the version before the defect being fixed
DIFF_CODE	The defect-related code we recognize from the content of the fixed file
COMMIT_DESCRIPTION	The description of the commit in the defect-related PR
COMMIT_TIME	The time of the commit in the defect-related PR
OLD_CONTENT	The content of the file version before defects being fixed
NEW_CONTENT	The content of the file version after defects being fixed
OLD_PATH	The old path of the changed files
NEW_PATH	The new path of the changed files
PR_TITLE	The title of the defect related PR
PR_DESCRIPTION	The description of the defect related PR

layer, and two linear layers, referring to the convolutional neural network (CNN) architectures. To simplify our modeling, each hidden layer had the same number of neurons. The numbers of layers and neurons were determined according to the results of the experiments. Once the neighborhood aggregation was completed, a polling layer and a softmax function were leveraged to predict the labels of samples, using the Adam optimization algorithm to minimize the cross-entropy loss accordingly for training the learning models. All GNN models were implemented in Python3.7 and PyTorch1.1.

Our baselines were five traditional classifiers based on 21 static file-level metrics. All metrics were calculated by the open-source tool *ck*.² In addition to CK Metrics, the datasets included the number of methods, the number of fields, the quantity of returns, and the quantity of loops. Models, including k -nearest neighbor (KNN) classifier, decision tree (DT) classifier, logistic regression (LR), random forest (RF), and AdaBoost (AB), were used to predict the defect proneness by learning these metrics. Previous studies have proven that these classifiers tend to achieve better results [26], [27]. We also implemented the cross-project prediction DBN-CP based on the semantic features as the state-of-the-art approach [7]. In the experiments, the proposed method was compared to five traditional baseline classifiers and the state-of-the-art approach based on the GHPR dataset.

In experiments, training and testing data were from the 6052 file-level learning instances of the GHPR dataset. Specifically, we employed 50× train-test-split to split data into a random training and a testing subset. For each time train-test-split, 4539 instances were selected for the training subset and 1513 instances were selected for the testing subset. The training subset was used to build the model, and the testing subsets provided

¹[Online]. Available: https://github.com/feiwu/GHPR_dataset²[Online]. Available: <https://github.com/mauricioaniche/ck>

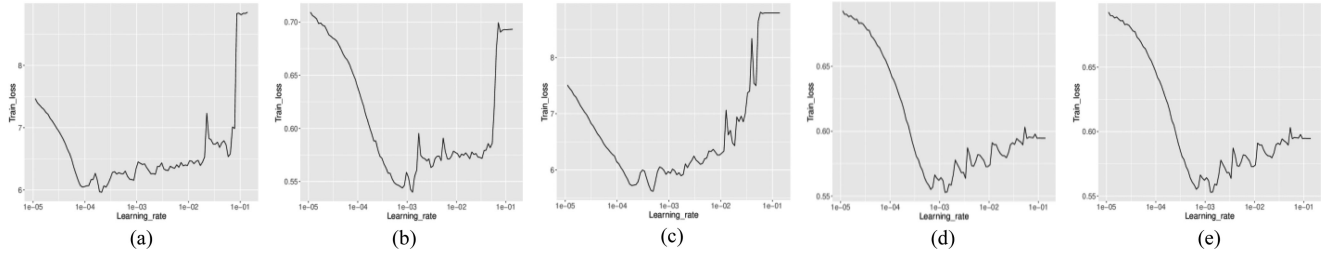


Fig. 7. Training loss for five GNNs with the LR range test. (a) GIN. (b) GCN. (c) SGC. (d) GAT. (e) SAGE.

TABLE III
CONFUSION MATRIX

	Predicted Negative	Predicted Positive
Actual Negative	True Positive	False Positive
Actual Positive	False Negative	True Negative

an unbiased evaluation for the performance of the model. We repeated the process of train-test-split $50\times$ and reported the average performance for every model.

D. Evaluation Metrics

Five measures were used to evaluate the effectiveness of the classification: accuracy, recall, precision, $F1$ score, and AUC score. Accuracy is the most common metric in the binary classification. However, $F1$ and AUC are also widely used in SDP due to the imbalance of datasets [28], [29]. As can be seen in Table III, the confusion matrix is counted by the actual labels and predicted labels.

The formulas for calculating the precision and recall are as follows:

$$\text{Precision} = \frac{\text{true positive}}{\text{true positive} + \text{false positive}} \quad (7)$$

$$\text{Recall} = \frac{\text{true positive}}{\text{true positive} + \text{false negative}}. \quad (8)$$

If the cost of a false positive is high, precision can effectively measure the performance of models. Recall focuses on situations where actual defective classes are predicted as non-defective classes. According to the following equation, $F1$ is a harmonic mean of precision and recall that seeks a balance of the two:

$$\mathcal{F}1 = \frac{2 \times \text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}}. \quad (9)$$

The receiver operating characteristic (ROC) curves are an effective way of visualizing the tradeoff between the true positive rate and false positive rate. In practice, the area under the ROC curve (i.e., AUC) is an excellent measure for summarizing the curve in a single value.

E. Hyperparameter Settings

Prior to commencing our experiments, we tuned five parameters, i.e., learning rate, number of layers, batch size, hidden size, and percentage of dropout. In the experiments of setting hyperparameters, we randomly selected 12.5% of the data (572

TABLE IV
RESULTS OF HYPERPARAMETERS OF GNNs

Model	Learning rate	N layers	Hidden_size	Dropout
GIN	0.0001	3	240	0.306
GCN	0.0005	7	244	0.694
SGC	0.00015	9	256	0.700
GAT	0.0005	6	222	0.301
SAGE	0.0005	3	178	0.300

instances in total) from the training set as the validation dataset. The remaining data were used to build the model and apply the model to the validation dataset. For reproducing the results, the fixed random seed was set to generating validation data and initializing model parameters.

1) *Learning Rate*: Once the type of loss function and optimizer are chosen, the learning rate becomes the critical parameter. Too small of a learning rate can exhibit an overfitting behavior and extend the convergence time. However, the optimal solution cannot be found with the excessive learning rate. One of the most popular tools for selecting the learning rate is the LR range test proposed by Smith *et al.* [30]. The core idea of the method is that the optimal learning rate is the maximum learning rate that can make the model converge. The learning rate was initialized to be $1e^{-5}$, which was then increased slowly. Using the LR range test of GCN as an example, Fig. 7 shows that losses failed to continue to decline until the learning rate reached $1e^{-4}$. As a result, the learning rate for the GCN model was selected as $1e^{-4}$.

2) *Model Parameters*: Limited to the experimental environment, the batch size was set to 256. Then, searching over the space of the remaining hyperparameters, including the number of layers, hidden size, and percentage of dropout, the search process attempted to find the maximum or minimum output of the learning model as a black-box function. A Bayesian optimization package was applied to develop a better exploration strategy [31]. The space of hyperparameters is as follows: $n_layers = \{1, 3, 5, 7, 9\}$, $hidden_size = \{32, 64, 128, 256, 512\}$, and $dropout = \{0.1, 0.3, 0.5, 0.7, 0.9\}$.

We randomly selected a combination as the initial parameter set. The next point to be explored was determined by the Bayesian optimization process. The accuracy of the five GNNs in 60 iterations is shown in Fig. 8, with point size and color for the hidden size and dropout. The results of the empirical hyperparameters that were set for each GNN are displayed in Table IV.

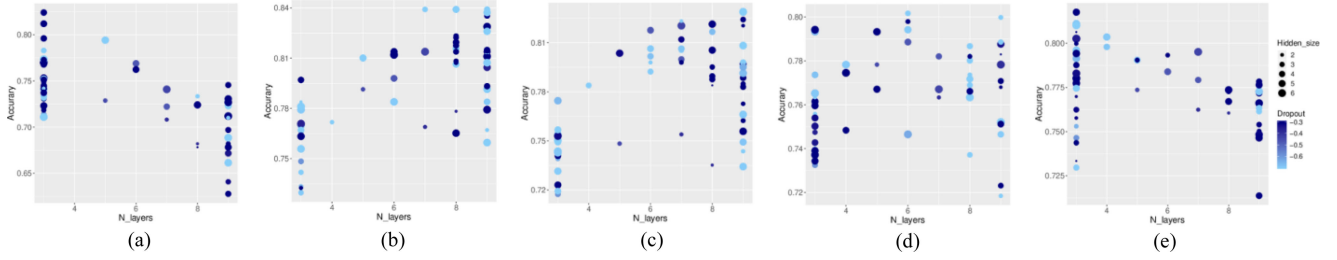


Fig. 8. Results for tuning hyperparameters of GNNs. (a) GIN. (b) GCN. (c) SGC. (d) GAT. (e) SAGE.

TABLE V
PREDICTIVE PERFORMANCE OF OUR PROPOSED APPROACH USING FIVE GNNs

Model	Accuracy	Precision	Recall	F1	AUC	Runtime
GIN	0.767	0.81	0.739	0.773	0.779	430.62
GCN	0.813	0.895	0.739	0.81	0.824	403.96
SGC	0.828	0.941	0.696	0.8	0.825	294.18
SAGE	0.781	0.882	0.652	0.75	0.781	346.41
GAT	0.732	0.889	0.696	0.78	0.802	368.44

F. Results and Analysis

1) *Answer to Q1: Which GNN Model Achieves the Best Performance?*: As a preliminary study, the experiment was designed to find out which GNN model achieves the best performance. The experimental results are reported in Table V, which compares GIN, GCN, SGC, SAGE, and GAT in terms of six evaluation metrics. This article adopted accuracy, precision, recall, $F1$ score, and AUC to evaluate the performance of models while reporting the runtime of a training process on a Nvidia 8 GB GTX 2080 graphics card. Each value was the average over the results of $50 \times$ train-test-split on the GHPR dataset. The highest values among the five GNNs appear in bold.

As can be seen in Table V, all five GNNs achieved well above 0.75 in accuracy, $F1$ score, and AUC score. In most cases, GCN and SGC were always among the top two methods in terms of the six metrics. For the $F1$ score, which is a tradeoff between precision and recall, GCN produced the best performance, with a value very close to that of SGC. Meanwhile, SGC obtained the highest average AUC value. GIN, SAGE, and GAT had average performance. No one model always resulted in the worst performance. The experiment indicates that GNNs can be applied to learning the representation of a defective subtree and achieved satisfactory results.

There was no significant gap between GCN and SGC in terms of AUC and $F1$ score. A clear benefit of the SGC methods was their efficiency, with the time costs of SGC models reduced by 25% compared with the runtime of GCN. Fig. 9 shows the loss value of 100 iterations for each GNN. The cross-entropy loss function was also defined to optimize our models. The figure indicates that all models show the loss convergence curves in 100 steps. SGC generated a better cross-entropy loss than other models after convergence. GCN and GAT had about 0.7 cross-entropy error. Based on the results, we selected SGC as the default GNN method in Q2 and Q3. Our proposed approach is named GNN-DP for the defect prediction based on GNNs.

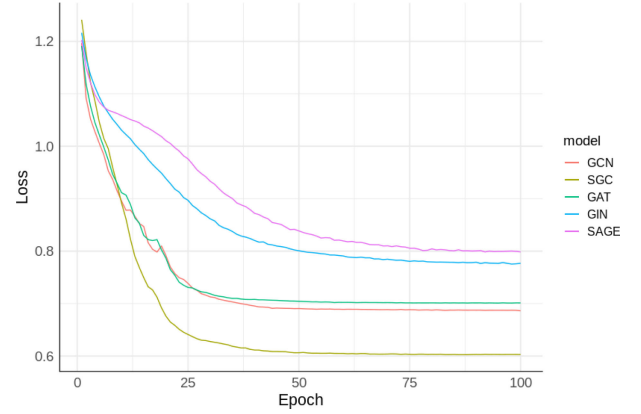


Fig. 9. Training loss curves for GNNs.

TABLE VI
RESULTS OF GNN-DP AND FIVE BASELINE APPROACHES

Model	Accuracy	Precision	Recall	F1	AUC
GNN-DP	0.828	0.941	0.696	0.800	0.825
AdaBoost	0.777	0.741	0.855	0.794	0.768
KNN	0.526	0.528	0.467	0.495	0.526
DT	0.728	0.718	0.746	0.732	0.719
LR	0.670	0.629	0.851	0.723	0.672
RF	0.803	0.758	0.828	0.791	0.803

2) *Answer to Q2: Does the Proposed Approach Improve the Performance of Traditional Machine Learning Models Based on Static Features?*: To validate the effectiveness of the proposed approach, we conducted an experiment to compare GNN-DP with five traditional machine learning models based on the static features. The five baselines included KNN classifier, DT classifier, the ensemble learning AB, LR, and RF. For the five classifiers, 21 static class-level metrics were used as the input of the models. The parameter of classifiers was estimated by a grid search implemented by the scikit-learn package. For GNN-DP, the SGC model aggregates the information of defective subtrees. The experimental results on the six models were given in Table VI. The best values among the models are highlighted in bold.

As given in Table VI, the proposed approach (GNN-DP) was superior to the five baseline approaches in four metrics. Compared with the traditional defect prediction, the GNN-DP approach improved by 19.4% on precision. However, there was no significant increase between our proposed approach and the others when it came to the indicator of recall. In terms of the

TABLE VII
INTERVAL SETTING OF CLIFF'S DELTA EXAMINATION

Effect size	Cliff's delta
Negligible	Cliff's delta < 0.147
Small	$0.147 \leq \text{Cliff's delta} < 0.33$
Medium	$0.33 \leq \text{Cliff's delta} < 0.474$
Large	$0.474 \leq \text{Cliff's delta}$

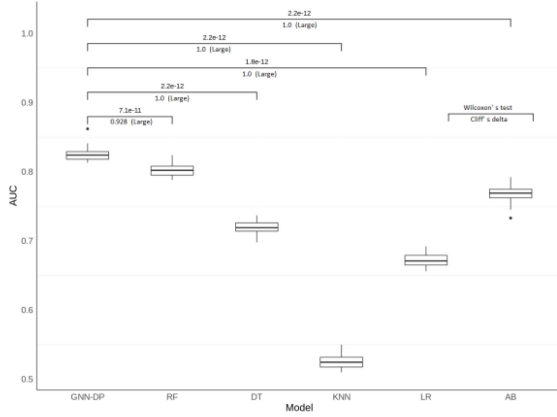


Fig. 10. Wilcoxon's test and Cliff's delta of GNN-DP and baseline approaches.

F1 score, which is the tradeoff between the two indicators of precision and recall, GNN-DP achieved the best *F1* score, at 0.800. We used Cliff's delta to examine the effect size, where the interval setting was set according to the study of Hess and Kromrey [32], as given in Table VII. Fig. 10 shows the boxplots of six models and Wilcoxon's test and the effect size of GNN-DP and the five baseline predictors in terms of AUC. It can be found that GNN-DP was the best performer in AUC, by achieving an average AUC of 0.825 and that LR was the worst performer of the six classifiers. We calculated the significance and magnitude of a difference between GNN-DP and baseline approaches, and added the annotation of *p* values and effect sizes to the plot in a single line. Wilcoxon's test indicated that, overall, GNN-DP produced defect prediction models that outperformed other baseline approaches at a 5% level of significance. Cliff's delta shows that the magnitude of the effect size between GNN-DP and other baseline predictors is large, which means that the performance difference between our approach and theirs is large enough.

3) *Answer to Q3: Does the Proposed Approach Outperform the State-of-the-Art DBN-CP?*: The purpose of this experiment was to compare the performance of our proposed GNN-DP with the state-of-the-art DBN-CP method. In the DBN model, the number of hidden layers was ten, and the size of each layer was 100. The data in Fig. 11 show that GNN-DP achieved comparable or better than the DBN-CP method. GNN-DP achieved higher accuracy and precision and showed a 30.7% improvement in the *F1* value. When using the AUC score as the metric, a 57.7% improvement was observed. We conducted Wilcoxon's sign test and Cliff's delta examination to show the significance and magnitude of differences between GNN-DP and DBN-CP, and added the annotation of statistical significance in each boxplot. Fig. 11 shows that the *p*-value was less than 0.05 and the effect size was large when the evaluation metrics were accuracy,

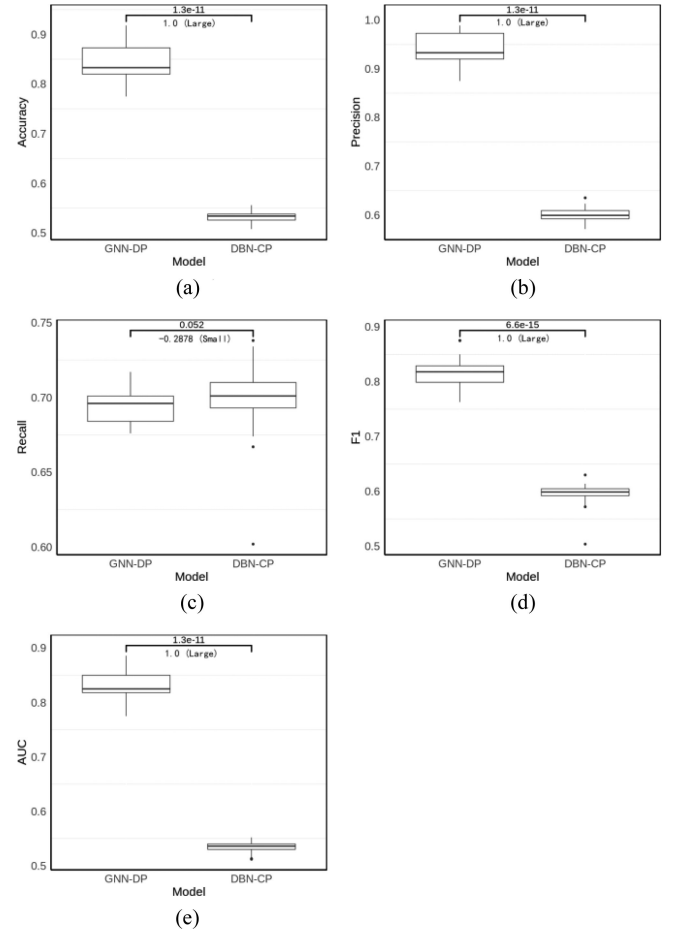


Fig. 11. Results of GNN-DP and DBN-CP. (a) Accuracy. (b) Precision. (c) Recall. (d) *F1*. (e) AUC.

precision, *F1*, and AUC. Hence, our proposed GNN-DP led to statistically significant improvements in these four metrics.

With the successive increases of other metrics, the recall indicator of our model was slightly lower than the DBN-CP method. That means that DBN-CP can catch a higher percentage of defective modules. However, the reduction of precision and accuracy reveals that DBN-CP arbitrarily classified most of the samples into defective samples.

In DBN-CP, semantic features are represented as the token vectors of the AST. The token vectors may be effective for the semantic of the module that contains defects. However, in many cases, defective semantic features are hidden by abundant non-defective information, so it is hard for DBN-CP to capture the actual defective characteristics. GNN-DP leverages the change information of the defect fix to obtain defective ASTs. The more accurate and relevant representation is learned by GNN models.

The abovementioned analysis suggests that our proposed approach led to better defect prediction performance. Considering that software defects are not only related to semantics, GNN-DP employed the code concepts to describe the macroscopic function and the environment around the defects. The following section shows how we validated the model with code concepts.

4) *Answer to Q4: How Effective is the Code Concepts Information?*: To shed light on this question, we conducted an

TABLE VIII
AUC OF FIVE GNNs WITH DIFFERENT TOPIC DIMENSIONS

Topic dimension	GIN	GCN	SGC	SAGE	GAT
0	0.74	0.717	0.798	0.698	0.75
30	0.779	0.824	0.825	0.781	0.802
50	0.895	0.925	0.929	0.786	0.853
70	0.77	0.821	0.803	0.717	0.779
90	0.748	0.817	0.837	0.786	0.807

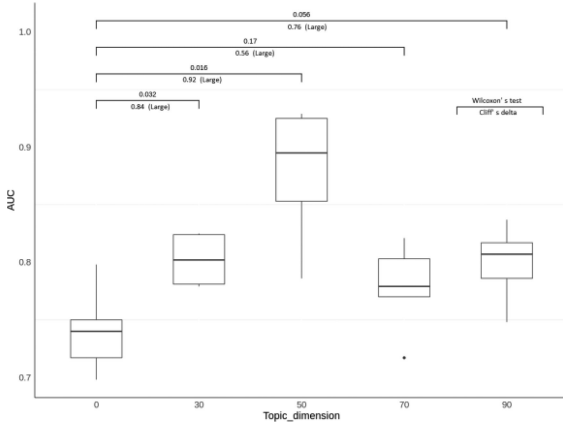


Fig. 12. Wilcoxon's test and Cliff's delta of different topic dimensions.

ablation study to measure the effect of code concepts information in the defective subtree. We fixed the other hyperparameters and compared the performance varied topic dimension = {0, 30, 50, 70, 90}. The models of the topic dimension of "0" constructed the control group without code concepts. The other number of topic dimensions indicated that code concepts were embedded into the vectors of a certain length by the word2vec model.

From the data in Table VIII, we can easily see the accuracy of the different GNNs with different sizes of concept embedding. Overall, GNN-DP achieved better results than the sample without code concepts. Using SGC as an example, the accuracy of GNN-DP with code concepts ranged from 0.819 to 0.858, which increased more than 7%. The accuracy first rose and then fell with the increase in the topic dimension. The models, except for SAGE, achieved the best performance with 50 dimensions of code concepts. The data also imply that SGC consistently performed with the highest accuracy on all topic dimensions. The results were confirmed with the answer to Q1, which suggested that SGC is a superior graph representation model for GNN-DP.

Fig. 12 shows the statistics of the AUC of the topic dimension at four levels and four results of Wilcoxon's significance test and Cliff's delta examination. The resulting p -value was 0.016 between the dimension of 50, and the control group indicated Wilcoxon's significance test passed statistically at a 5% significance level. The resulting Cliff's delta was 0.92 between the dimension of 50, and the control group indicated the effect size was large. The accuracy had a small drop when the dimension exceeded 50. Note that there are fewer than 40,000 words in our topic dictionary. Due to the limitation of the size of the topic dictionary, the word embedding with too large topic dimension tended to overfit. This experiment showed that the code concepts can help us identify the pattern of subtrees that are defect-prone.

V. THREATS TO VALIDITY

Below, we discuss some potential threats to the validity of the obtained results.

A. GHPR Dataset

Our experiments depended on the GHPR dataset we collected in PRs based on the GitHub workflow. Because the GHPR dataset is original and has not been widely validated, it may include some nondefective noise data or incorrect project information and fix-inducing change. Other programming languages besides Java should be included in this dataset, although an AST is a common representation in many languages. More experiments need to be conducted, based on our GHPR dataset, to eliminate this impediment. In the future, we hope to maintain a high-quality GHPR dataset.

B. Reimplementation of State-of-the-Art DBN-CP

First, there was a lack of systematic comparison of the existing defect prediction methods based on semantic metrics. Because many existing studies use different datasets, the experimental results cannot be directly compared. Therefore, DBN-CP is the best performer only in terms of using semantic information within our known range. Second, since the source code of the state-of-the-art DBN-CP was not opened, we reimplemented the DBN model to obtain the semantic representations from the source code and the change information. It is difficult to control each variable and each model parameter in the state-of-the-art approach. Fortunately, the PROMISE datasets used in the DBN-CP model are publicly available, so this article evaluated the DBN-CP reimplementation using PROMISE datasets. TABLE IX indicates that our reimplementation had a comparable $F1$ scores performance, as reported in [7].

VI. RELATED WORK

A. Defect Prediction

Previous work on the traditional SDP has focused on datasets, metrics, preprocessing, and classifiers. NASA datasets are composed of 13 software programs, which are publicly available and widely used [33]. Zimmermann *et al.* [34] mapped the defects to components and contributed to the Eclipse bug dataset. Although the available datasets serve as benchmarks and support the development of SDP, the quality of these datasets attracts more attention [35]. Researchers have shown that resampling technique and feature selection have a significant positive effect on predict defects. Bennin *et al.* [36] leveraged a synthetic oversampling approach MAHAKIL, which was superior to the other oversampling methods on 20 imbalanced datasets. Khoshgoftaar *et al.* [37] ranked features based on the random undersampling and iterative selection to improve the performance of classifiers. For metrics, the traditional SDP holds that defects are related to module size and complexity. Various metrics include McCabe's cyclomatic complexity [38], Halstead complexity measures [39], CK Metric [40], and MOOD [41]. For discarding

TABLE IX
COMPARISON OF THE F1 SCORES OF REIMPLEMENTATION

Source	Target	Original	Reimplementation
camel1.4	ant1.6	97.9	69.6
poi3.0	ant1.6	47.8	68.5
camel1.2	ant1.7	31.2	32.5
jEdit3.2	ant1.7	41.7	36.7
ant1.6	camel1.4	31.6	52.4
jEdit4.1	camel1.4	69.3	60
ant1.5	camel1.6	49	48.4
lucene2.0	camel1.6	49.2	54.3
xerces1.2	jEdit4.0	51.9	39.2
camel1.4	jEdit4.1	61.5	40.3
log4j1.1	jEdit4.1	50.3	58.4
jEdit4.1	log4j1.1	64.5	70.3
lucene2.2	log4j1.1	61.8	73
xalan2.5	lucene2.2	59.4	58
log4j1.1	lucene2.2	59.2	60.1
poi2.5	lucene2.4	64.9	65
xalan2.4	lucene2.4	61.6	66.5
lucene2.2	xalan2.5	55	65
xerces1.3	xalan2.5	57.2	65.3
xalan2.5	xerces1.3	38.6	43.5
poi2.5	synapse1.1	42.5	58.3
poi3.0	synapse1.2	51.4	43.3
synapse1.2	poi3.0	66.1	60.4
ant1.6	poi3.0	61.9	63.2
synapse1.1	poi2.5	44.6	42.7
ant1.6	poi2.5	47.5	50.8

redundant and irrelevant metrics, we proposed a novel cluster-based feature selection method by defining a feature quality coefficient [42]. Lessmann *et al.* [43] compared 22 classifiers in terms of AUC on NASA's MDP dataset. Experimenters found that simple classifiers can capture the relationship between the traditional static metrics and software defects. Hence, finding a better representation of software defects than static metrics is the key factor for SDP.

B. Defect Prediction on Source Code

The goal of this part of the experiment was to leverage machine learning on source code. Recent accomplishments have had a significant impact in providing software engineering solutions, such as defect prediction [9] and repair [44], code summarization [45], and clone detection. The machine learning framework on the source code can be divided into three categories: code-generating models, representational models, and pattern mining models [8]. Mou *et al.* [10] employed tree-based CNNs for programming language processing. More evidence and numerous applications have followed for SDP. Wang *et al.* [7] extracted token vector from ASTs and proposed a file-level and change-level defect prediction model, which was evaluated through improvements in the within-project prediction and cross-project prediction. A tree-based long short-term memory recurrent neural network (RNN) [9] and an attention-based RNN [46] were also proposed to learn semantic features in ASTs. In addition, the control flow graph was explored to automatically learn defect features by CNN [14]. Unlike the abovementioned methods, our proposed approach focuses on the preprocessing of semantic structure and integrates the information about code concepts.

VII. CONCLUSION

In this article, we presented GNN-DP, a novel approach to constructing an end-to-end automation framework that learns the latent defective representation from code snippets, project information, and fix-inducing change. Our approach differs from the traditional metrics in that defective fragments are represented as a syntactic tree-based structure to preserve the defect information. GNN-DP employs GNNs to aggregate semantic information and to predict defective modules. In addition, our approach is unique in that it combines semantic information with code concepts to capture the defect patterns in specific environments.

The performance of our proposed approach was validated in terms of five well-known measures: accuracy, precision, recall, F1 score, and AUC. The empirical dataset used 6052 real-world defects from 307 projects, which we collected from GitHub. Comparing our proposed approach with five baseline models and the state-of-the-art approach, we observed significant improvement in the performance of our approach. Wilcoxon's signed-rank tests and Cliff's delta examination showed that code concepts are effective in SDP.

Future work will focus on finding a better method of pruning defective ASTs using the fix-inducing change and delving deeper into the mechanism of graph representation learning.

REFERENCES

- [1] S. Wang and X. Yao, "Using class imbalance learning for software defect prediction," *IEEE Trans. Rel.*, vol. 62, no. 2, pp. 434–443, Jun. 2013.
- [2] K. O. Elish and M. O. Elish, "Predicting defect-prone software modules using support vector machines," *J. Syst. Softw.*, vol. 81, no. 5, pp. 649–660, 2008.
- [3] X. Bai, H. Zhou, and H. Yang, "An HVSM-based GRU approach to predict cross-version software defects," *Int. J. Perform. Eng.*, vol. 16, no. 6, pp. 979–990, 2020.
- [4] Y. Zhou *et al.*, "How far we have progressed in the journey? An examination of cross-project defect prediction," *ACM Trans. Softw. Eng. Methodol.*, vol. 27, no. 1, pp. 1–51, 2018.
- [5] E. Ceylan, F. O. Kutlubay, and A. B. Bener, "Software defect identification using machine learning techniques," in *Proc. 32nd EUROMICRO Conf. Softw. Eng. Adv. Appl.*, 2006, pp. 240–247.
- [6] H. Tong, B. Liu, and S. Wang, "Software defect prediction using stacked denoising autoencoders and two-stage ensemble learning," *Inf. Softw. Technol.*, vol. 96, pp. 94–111, 2018.
- [7] S. Wang, T. Liu, J. Nam, and L. Tan, "Deep semantic feature learning for software defect prediction," *IEEE Trans. Softw. Eng.*, to be published.
- [8] M. Allamanis, E. T. Barr, P. Devanbu, and C. Sutton, "A survey of machine learning for big code and naturalness," *ACM Comput. Surv.*, vol. 51, no. 4, 2018, Art. no. 81.
- [9] H. K. Dam, T. Tran, T. T. M. Pham, S. W. Ng, J. Grundy, and A. Ghose, "Automatic feature learning for predicting vulnerable software components," *IEEE Trans. Softw. Eng.*, to be published.
- [10] L. Mou, G. Li, L. Zhang, T. Wang, and Z. Jin, "Convolutional neural networks over tree structures for programming language processing," in *Proc. 30th AAAI Conf. Artif. Intell.*, 2016, pp. 1287–1293.
- [11] Q. Song, Z. Jia, M. Shepperd, S. Ying, and J. Liu, "A general software defect-proneness prediction framework," *IEEE Trans. Softw. Eng.*, vol. 37, no. 3, pp. 356–370, May/Jun. 2011.
- [12] K. Xu, W. Hu, J. Leskovec, and S. Jegelka, "How powerful are graph neural networks?," in *Proc. 7th Int. Conf. Learn. Representation*, 2019, pp. 1–17.
- [13] J. Zhang, X. Wang, H. Zhang, H. Sun, K. Wang, and X. Liu, "A novel neural source code representation based on abstract syntax tree," in *Proc. IEEE/ACM 41st Int. Conf. Softw. Eng.*, 2019, pp. 783–794.
- [14] A. V. Phan, M. L. Nguyen, and L. T. Bui, "Convolutional neural networks over control flow graphs for software defect prediction," in *Proc. IEEE 29th Int. Conf. Tools Artif. Intell.*, 2017, pp. 45–52.

- [15] M. White, M. Tufano, C. Vendome, and D. Poshvanyk, "Deep learning code fragments for code clone detection," in *Proc. 31st IEEE/ACM Int. Conf. Autom. Softw. Eng.*, 2016, pp. 87–98.
- [16] Z. Cai, L. Lu, and S. Qiu, "An abstract syntax tree encoding method for cross-project defect prediction," *IEEE Access*, vol. 7, pp. 170844–170853, 2019.
- [17] J.-R. Falleri, F. Morandat, X. Blanc, M. Martinez, and M. Monperrus, "Fine-grained and accurate source code differencing," in *Proc. 29th ACM/IEEE Int. Conf. Autom. Softw. Eng.*, 2014, pp. 313–324.
- [18] V. D. Blondel, J.-L. Guillaume, R. Lambiotte, and E. Lefebvre, "Fast unfolding of communities in large networks," *J. Statist. Mech. Theory Exp.*, vol. 2008, no. 10, 2008, Art. no. P10008.
- [19] T.-H. Chen, S. W. Thomas, M. Nagappan, and A. E. Hassan, "Explaining software defects using topic models," in *Proc. IEEE Int. Working Conf. Mining Softw. Repositories*, 2012, pp. 189–198.
- [20] T. Mikolov, K. Chen, G. Corrado, and J. Dean, "Efficient estimation of word representations in vector space," in *Proc. 1st Int. Conf. Learn. Representations*, 2013, pp. 1301–1378.
- [21] T. N. Kipf and M. Welling, "Semi-supervised classification with graph convolutional networks," in *Proc. 5th Int. Conf. Learn. Representations*, 2017, pp. 1–14.
- [22] F. Wu, T. Zhang, A. H. de Souza, C. Fifty, T. Yu, and K. Q. Weinberger, "Simplifying graph convolutional networks," in *Proc. 36th Int. Conf. Mach. Learn.*, 2019, pp. 6861–6871.
- [23] P. Velićković, A. Casanova, P. Liò, G. Cucurull, A. Romero, and Y. Bengio, "Graph attention networks," in *Proc. 6th Int. Conf. Learn. Representations*, 2018, pp. 1–12.
- [24] W. L. Hamilton, R. Ying, and J. Leskovec, "Inductive representation learning on large graphs," in *Proc. 31st Int. Conf. Neural Inf. Process. Syst.*, 2017, pp. 1025–1035.
- [25] J. Xu, L. Yan, F. Wang, and J. Ai, "A GitHub-based data collection method for software defect prediction," in *Proc. 6th IEEE Int. Conf. Dependable Syst. Appl.*, 2020, pp. 100–108.
- [26] B. Ghotra, S. McIntosh, and A. E. Hassan, "Revisiting the impact of classification techniques on the performance of defect prediction models," in *Proc. IEEE/ACM 37th Int. Conf. Softw. Eng.*, vol. 1, 2015, pp. 789–800.
- [27] C. Tantithamthavorn, S. McIntosh, A. E. Hassan, and K. Matsumoto, "An empirical comparison of model validation techniques for defect prediction models," *IEEE Trans. Softw. Eng.*, vol. 43, no. 1, pp. 1–18, Jan. 2017.
- [28] Y. Yang, J. Ai, and F. Wang, "Defect prediction based on the characteristics of multilayer structure of software network," in *Proc. IEEE Int. Conf. Softw. Qual., Rel. Secur. Companion*, 2018, pp. 27–34.
- [29] W. Fu and T. Menzies, "Revisiting unsupervised learning for defect prediction," in *Proc. 11th Joint Meeting Found. Soft. Eng.*, 2017, pp. 72–83.
- [30] S. L. Smith, P.-J. Kindermans, C. Ying, and Q. V. Le, "Don't decay the learning rate, increase the batch size," in *Proc. 6th Int. Conf. Learn. Representations*, 2018.
- [31] J. Snoek, H. Larochelle, and R. P. Adams, "Practical Bayesian optimization of machine learning algorithms," in *Proc. 25th Int. Conf. Neural Inf. Process. Syst.*, 2012, pp. 2951–2959.
- [32] M. R. Hess and J. D. Kromrey, "Robust confidence intervals for effect sizes: A comparative study of Cohen's d and Cliff's delta under non-normality and heterogeneous variances," in *Proc. Annu. Meeting Amer. Educ. Res. Assoc.*, 2004, pp. 1–30.
- [33] H. Hata, T. Kikuno, and O. Mizuno, "A systematic review of software fault prediction studies and related techniques," *Comput. Softw.*, vol. 29, no. 1, pp. 106–117, 2012.
- [34] T. Zimmermann, R. Premraj, and A. Zeller, "Predicting defects for eclipse," in *Proc. 3rd Int. Workshop Predictor Models Softw. Eng.*, 2007, p. 9.
- [35] M. Shepperd, Q. Song, Z. Sun, and C. Mair, "Data quality: Some comments on the NASA software defect datasets," *IEEE Trans. Softw. Eng.*, vol. 39, no. 9, pp. 1208–1215, Sep. 2013.
- [36] K. E. Bennin, J. Keung, P. Phannachitta, A. Monden, and S. Mensah, "MAHAKIL: Diversity based oversampling approach to alleviate the class imbalance issue in software defect prediction," *IEEE Trans. Softw. Eng.*, vol. 44, no. 6, pp. 534–550, Jun. 2018.
- [37] T. M. Khoshgoftaar, K. Gao, A. Napolitano, and R. Wald, "A comparative study of iterative and non-iterative feature selection techniques for software defect prediction," *Inf. Syst. Frontiers*, vol. 16, no. 5, pp. 801–822, 2014.
- [38] T. J. McCabe, "A complexity measure," *IEEE Trans. Softw. Eng.*, vol. SE-2, no. 4, pp. 308–320, Dec. 1976.
- [39] M. H. Halstead, *Elements of Software Science*. New York, NY, USA: Elsevier, 1977.
- [40] S. R. Chidamber and C. F. Kemerer, "A metrics suite for object oriented design," *IEEE Trans. Softw. Eng.*, vol. 20, no. 6, pp. 476–493, Jun. 1994.
- [41] V. R. Basili, L. C. Briand, and W. L. Melo, "A validation of object-oriented design metrics as quality indicators," *IEEE Trans. Softw. Eng.*, vol. 22, no. 10, pp. 751–761, Oct. 1996.
- [42] F. Wang, J. Ai, and Z. Zou, "A cluster-based hybrid feature selection method for defect prediction," in *Proc. IEEE 19th Int. Conf. Softw. Qual., Rel. Secur.*, 2019, pp. 1–9.
- [43] S. Lessmann, B. Baesens, C. Mues, and S. Pietsch, "Benchmarking classification models for software defect prediction: A proposed framework and novel findings," *IEEE Trans. Softw. Eng.*, vol. 34, no. 4, pp. 485–496, Jul./Aug. 2008.
- [44] J. Jiang, Y. Xiong, H. Zhang, Q. Gao, and X. Chen, "Shaping program repair space with existing patches and similar code," in *Proc. 27th ACM SIGSOFT Int. Symp. Softw. Testing Anal.*, 2018, pp. 298–309.
- [45] X. Hu, G. Li, X. Xia, D. Lo, S. Lu, and Z. Jin, "Summarizing source code with transferred API knowledge," in *Proc. 27th Int. Joint Conf. Artif. Intell.*, 2018, pp. 2269–2275.
- [46] G. Fan, X. Diao, H. Yu, K. Yang, and L. Chen, "Software defect prediction via attention-based recurrent neural network," *Sci. Program.*, vol. 2019, 2019, Art. no. 6230953.



Jiayi Xu received the B.S. degree in safety engineering from the School of Reliability and System Engineering, Beihang University, Beijing, China, in 2019.

His research interests include machine learning, software defect prediction, and deep learning.



Fei Wang received the M.S. degree in industrial engineering from the School of Reliability and System Engineering, Beihang University, Beijing, China, in 2020.

His research interests include software defect prediction, software data mining, and deep learning.



Jun Ai received the Ph.D. degree in engineering from the School of Reliability and System Engineering, Beihang University, Beijing, China, in 2006.

He is currently an Associate Professor with the School of Reliability and Systems Engineering, Beihang University, Beijing. His research interests include software reliability, software defect prediction, software quality evaluation, and software networks.